NAT Router Lite

Funcionamiento, arquitectura y decisiones técnicas

Documentación del proyecto

Base técnica: ESP-IDF 6.1
ESP32 y ESP32-S3

Resumen

NAT Router Lite convierte un ESP32 o ESP32-S3 en un router Wi-Fi IPv4 con configuración mediante navegador. El dispositivo mantiene un punto de acceso para sus clientes y una estación conectada al módem, guarda las credenciales en NVS y comparte la salida mediante NAPT. Este documento describe el código actual, sus interfaces y las razones técnicas de su organización: eventos serializados, DNS local estable, confirmación HTTP, diagnóstico de errores e indicador configurable. También delimita qué comprueba el estado mostrado y qué funciones todavía no existen. Las explicaciones de diseño se basan en el comportamiento observable del código; no constituyen mediciones de rendimiento.

Índice

1. Objetivo y recorrido de uso **3**

2. Topología y separación entre control y tráfico **3**

3. Organización del código **4**

4. Inicialización y modelo de concurrencia **4**

4.1. Una tarea coordina las operaciones del controlador 5

4.2. Reconexión y actualización de credenciales 5

5. Credenciales: validación y persistencia **6**

5.1. Interpretación correcta del formulario 6

5.2. Qué se guarda y cuándo se conserva 6

6. IPv4, DHCP y NAPT **7**

7. Proxy DNS: la razón del DNS local estable **8**

7.1. Por qué cambiar el DNS de DHCP no bastaba 8

7.2. Selección del servidor de origen 8

7.3. Recorrido de una consulta 8

7.4. Límites, errores y cambios de conexión 9

8. Servidor HTTP y aplicación del navegador **10**
8.1. Recursos y contrato de las rutas . . . 10
8.2. Estado publicado y significado de cada campo . . . 10
8.3. Sondeo automático y presentación . . . 11

9. Diagnóstico de asociación y autenticación **11**

10. Indicador y opciones específicas del hardware **12**
10.1. Por qué se utiliza RMT para el RGB . . . 13
10.2. FTM y consola USB . . . 13

11. Perfiles de ESP-IDF y compilación reproducible **13**
11.1. Chip, placa, defaults y configuración generada . . . 13
11.2. Comandos de trabajo . . . 14

12. Presupuesto de recursos y tratamiento de fallos **15**

13. Seguridad y alcance real de la aplicación **15**

14. Verificación y diagnóstico práctico **16**
14.1. Qué evidencia existe . . . 16
14.2. Orden útil para localizar un problema . . . 16

15. Cómo generar y mantener este documento **17**

1. Objetivo y recorrido de uso

El objetivo es permitir que uno o varios clientes utilicen la conexión Wi-Fi de un módem a través del ESP, sin recompilar cada vez que cambien las credenciales del módem. El firmware crea la red `ESP32-NAT`, con clave inicial `12345678`, y expone su aplicación en `http://192.168.2.1`. El límite configurado es de tres clientes asociados al AP. Tanto el ESP32 como el ESP32-S3 de este proyecto trabajan con redes de 2,4 GHz.

El primer uso sigue esta secuencia:

1. El usuario conecta el teléfono al AP del ESP. Si todavía no hay salida a Internet, debe mantener esa conexión para configurar el equipo.
2. Abre la dirección HTTP anterior e introduce el SSID y la clave de la red del módem. El botón con un ojo permite mostrar u ocultar la clave.
3. El servidor valida y guarda los datos. Una redirección conduce a una página que confirma la recepción y consulta el estado automáticamente.
4. El ESP intenta asociarse, obtener una IPv4 y preparar NAPT y DNS. La página muestra espera, un diagnóstico o el estado preparado.
5. Al reiniciar, se cargan las credenciales guardadas. Para cambiarlas se utiliza el enlace de la página; no es necesario borrar NVS.

Guardar datos, conectarse a una red y disponer de Internet son hechos distintos. El guardado puede ser correcto aunque la contraseña sea incorrecta para ese módem. Una asociación correcta puede carecer de IPv4 por un problema DHCP. Incluso con IPv4, NAT y DNS preparados, el módem puede haber perdido su servicio de Internet. La aplicación informa su estado local y las razones comunicadas por el controlador; no realiza una prueba activa contra Google, YouTube u otro servicio externo.

2. Topología y separación entre control y tráfico

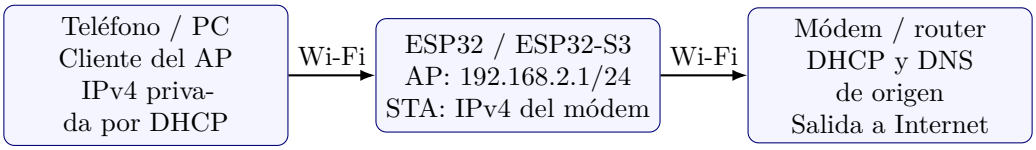


Figura 1: Dos interfaces lógicas sobre una radio Wi-Fi. Las flechas representan el recorrido de salida.

Plano de control. El navegador habla con el servidor HTTP local, que valida formularios, solicita almacenamiento y publica estado. Ese intercambio termina en el ESP.

Plano de datos. El tráfico de las aplicaciones del teléfono atraviesa lwIP y NAPT hacia la interfaz STA. Los paquetes de YouTube o Google no se retransmiten mediante el servidor HTTP del proyecto. El proxy DNS sí termina consultas UDP locales y las envía de nuevo al servidor DNS seleccionado.

El modo `WIFI_MODE_APSTA` mantiene AP y STA simultáneamente. No equivale a disponer de dos radios independientes: el canal de la red a la que se asocia la STA tiene prioridad y el AP debe acompañarlo [1]. Por ello, el canal inicial 1 del AP no es una promesa de que permanecerá en 1. El cambio de canal puede afectar transitoriamente a clientes que no sigan adecuadamente el anuncio del AP.

Razón de diseño. APSTA permite conservar el acceso a la configuración mientras la conexión al módem falla o se modifica. NAPT permite usar una subred propia sin exigir rutas adicionales en el módem. A cambio, el resultado es un router de capa IP con traducción de puertos, no un puente transparente de capa 2. El rendimiento comparte los recursos de radio, CPU y memoria del mismo microcontrolador.

3. Organización del código

Unidad	Responsabilidad	Dependencias propias
`main`	Arranque y actualización del indicador	NVS, Wi-Fi, HTTP, LED
`nvs`	Leer y guardar credenciales	Ninguna
`wifi`	AP, STA, DHCP, eventos, NAPT, reconexión, diagnóstico y parser	NVS, proxy DNS
`server`	HTTP y recursos de la aplicación	Wi-Fi y su parser público
`dns_proxy`	Reenvío DNS UDP	Ninguna
`status_led`	Indicador GPIO o RGB mediante RMT	Ninguna

Cada componente declara en CMake las dependencias de ESP-IDF que utiliza. Los encabezados públicos están en `include/`; `components/wifi/wifi_internal.h` permanece privado. Dentro de Wi-Fi, `ap.c` configura AP/DHCP, `sta.c` configura la estación y clasifica fallos, `wifi.c` coordina eventos y estado, y `wifi_form.c` decodifica el formulario.

`main/main.c` inicializa LED, NVS, Wi-Fi y servidor en ese orden. Después consulta el estado de conexión para actualizar el LED cada 500 ms. No contiene manejadores HTTP, parser, lógica DNS ni el flujo completo de reconexión.

Razón de diseño. La división sigue responsabilidades y dependencias, de modo que el servidor solicita acciones a Wi-Fi sin manipular sus variables internas ni abrir NVS directamente. DNS conserva una tarea y un protocolo propios; el LED solo recibe un booleano. Estas fronteras permiten probar el parser o el proxy en host y cambiar una presentación sin trasladar la lógica de red a la interfaz.

El almacenamiento expone `router_nvs.h`, en lugar de un encabezado local llamado `nvs.h`. Así se evita ocultar el encabezado homónimo del SDK durante la resolución de inclusiones. El namespace y las claves no se cambiaron al modularizar: se conserva compatibilidad con las credenciales que ya estaban guardadas.

La API pública Wi-Fi incluye `router_wifi_start()`, `router_wifi_get_status()`, `router_wifi_is_connecte` `router_wifi_store_credentials()` y `router_wifi_apply_credentials()`. Guardar y aplicar son operaciones separadas: la primera puede fallar en flash; la segunda puede fallar al encolar el trabajo. La respuesta HTTP puede distinguir ambas situaciones.

4. Inicialización y modelo de concurrencia

El arranque de Wi-Fi realiza las siguientes operaciones:

1. Crea el temporizador de reconexión antes de habilitar los eventos.
2. Inicializa ESP-NETIF, el bucle de eventos y sus manejadores.

3. Inicializa el controlador y selecciona APSTA.
4. Crea las interfaces AP y STA, configura DHCP y carga credenciales.
5. Inicia el listener DNS en la dirección del AP.
6. Ejecuta `esp_wifi_start()` y espera su retorno correcto.
7. Publica `ROUTER_CONNECT`; el manejador de eventos inicia entonces la conexión STA.

El listener DNS existe antes de que un teléfono pueda obtener su primera concesión DHCP. Por otra parte, la primera conexión se solicita después de que termine `esp_wifi_start()`. En las pruebas originales del S3, conectar directamente al recibir `WIFI_EVENT_STA_START` llegó a producir `ESP_ERR_WIFI_NOT_STARTED`. La secuencia actual evita esa carrera de arranque en este proyecto; no significa que el patrón de conectar desde ese evento esté prohibido en todos los programas de IDF.

4.1. Una tarea coordina las operaciones del controlador

El callback del temporizador publica un evento `ROUTER_CONNECT`; no ejecuta directamente `esp_wifi_connect()`. El manejador HTTP publica `ROUTER_UPDATE_CREDENTIALS` mediante la API pública. El bucle de eventos procesa esas solicitudes y las notificaciones Wi-Fi/IP.

Razón de diseño. Serializar las operaciones de conexión, desconexión y cambio de configuración reduce carreras entre la tarea HTTP, la tarea del temporizador y el controlador. El callback del temporizador queda corto y el servidor puede contestar sin esperar toda la asociación. Esto no convierte el sistema en un programa de una sola tarea: HTTP, DNS, temporizadores, Wi-Fi y lwIP siguen teniendo ejecución concurrente.

Los indicadores compartidos con HTTP utilizan atómicos C11: conectado, credenciales guardadas, NAPT, conflicto y última razón de desconexión. Los campos se leen individualmente; la estructura resultante no es una fotografía transaccional de todos ellos. Durante una transición una consulta puede observar una combinación pasajera que se estabiliza en el siguiente sondeo. Las banderas privadas que gobiernan la aplicación de credenciales se usan desde la tarea de eventos.

4.2. Reconexión y actualización de credenciales

Tras una desconexión se marca la salida como no disponible: se limpia el estado de conexión, se deshabilita NAPT y se invalida el DNS de origen. Un temporizador de una sola ejecución programa un nuevo intento después de tres segundos. A esto se añaden el escaneo de canales y los reintentos internos del controlador, configurados con `failure_retry_cnt = 2`. Tres segundos es el intervalo programado entre intentos de la aplicación, no la duración total máxima de una conexión.

Al recibir nuevas credenciales, el flujo es:

1. Confirmar el guardado en NVS y limpiar el diagnóstico anterior.
2. Encolar la actualización, marcar la salida sin servicio y detener el temporizador que pudiera estar pendiente.
3. Solicitar desconexión STA, recuperar los datos guardados y aplicar la nueva configuración. Si todavía no se puede, mantener la actualización pendiente para un intento posterior.
4. Programar la conexión con la configuración actualizada.
5. Al obtener IPv4, preparar la ruta, NAPT y el DNS de origen.

La desconexión provocada localmente por cambiar claves no debe mostrarse como si el módem hubiera rechazado la nueva contraseña. El código reconoce las razones de salida local esperadas y evita convertirlas en un fallo de autenticación. Un fallo real posterior sí queda registrado.

`WIFI_EVENT_STA_CONNECTED` indica asociación, pero el booleano `connected` se activa al recibir `IP_EVENT_STA_GOT_IP`. `STA_STOP`, `STA_DISCONNECTED` y `STA_LOST_IP` invalidan el estado de salida correspondiente. Si aparece una IP mientras hay un cambio de credenciales pendiente, no se presenta como la conexión final de esa actualización.

5. Credenciales: validación y persistencia

5.1. Interpretación correcta del formulario

El navegador envía `ssid` y `password` mediante un POST con codificación de formulario URL. El servidor acumula todos los fragmentos del cuerpo: una lectura de socket no garantiza recibir el mensaje entero. Se rechaza un cuerpo vacío, de 320 bytes o más, con NUL embebido o con campos inválidos. El búfer de 320 bytes contempla la expansión de cada byte a tres caracteres `%xx` y la sintaxis de ambos campos.

El parser separa primero los pares por `&` y cada clave de su valor por `=`; después decodifica. Un separador codificado dentro de una contraseña debe seguir siendo parte de su valor. Por ejemplo:

```
ssid=Mi+red&password=abc%26def%3D123
# SSID decodificado: Mi red
# Clave decodificada: abc&def=123
```

El signo `+` significa espacio; `%2B` representa un signo más literal. Las secuencias incompletas, no hexadecimales o que produzcan `%00` son inválidas. Los dos campos requeridos deben aparecer una sola vez. Las claves desconocidas bien formadas se ignoran; no sustituyen a los campos obligatorios.

Se aceptan SSID de 1 a 32 bytes y claves de 8 a 63 bytes, o una PSK de 64 caracteres hexadecimales. Los búferes de salida tienen 33 y 65 bytes para incluir el terminador NUL. Son límites en bytes, no en caracteres Unicode: un carácter acentuado puede ocupar más de un byte UTF-8. Los atributos HTML ayudan al usuario, pero la validación definitiva se realiza en C. No hay un modo de formulario para redes abiertas.

Razón de diseño. Validar en el servidor evita depender de un navegador concreto o de sus límites de formulario. Separar antes de decodificar conserva contraseñas que contienen `&`, `=` o espacios. El parser es C independiente de ESP-IDF y no modifica sus salidas si falla, por lo que puede probarse con casos de borde en un ordenador.

5.2. Qué se guarda y cuándo se conserva

El componente NVS utiliza la partición NVS predeterminada, el namespace `wifi` y las claves de cadena `ssid` y `password`. Abre en lectura/escritura, escribe ambas cadenas y llama a `nvs_commit()`; solo informa éxito si todas esas operaciones devuelven `ESP_OK`. Al cargar usa lectura solamente y comprueba que las cadenas caben y cumplen las longitudes mínimas esperadas. NVS es el almacenamiento de pares clave–valor en flash de ESP-IDF [3].

El arranque normal no borra credenciales. La inicialización recupera automáticamente la partición mediante borrado únicamente si recibe `ESP_ERR_NVS_NO_FREE_PAGES` o `ESP_ERR_NVS_NEW_VERSION_FOUND`. Otros fallos se propagan al arranque. Si no hay datos válidos, la STA usa los marcadores `SSID`/`PASS` y el AP permanece disponible para recibir los

datos reales. Es posible ver avisos de red no encontrada antes de configurar el equipo.

Guardar dos claves y hacer commit no añade por sí mismo una transacción de aplicación que garantice una pareja indivisible ante cualquier corte de energía o error intermedio. El código actual no utiliza un registro versionado ni un único blob que agrupe SSID y clave. Si se necesitara esa garantía adicional, habría que diseñar y validar ese formato de persistencia.

Razón de diseño. Dos cadenas pequeñas encajan con NVS y conservan compatibilidad con los datos de la versión anterior. No se escribe flash en cada sondeo de estado ni en cada actualización del LED: se guarda cuando se reciben credenciales. La información sensible no se incorpora al JSON de estado ni a la página de confirmación.

6. IPv4, DHCP y NAPT

El AP tiene dirección y gateway `192.168.2.1`, con máscara `255.255.255.0`. Su servidor DHCP entrega configuración a los clientes. La interfaz STA utiliza el DHCP de la red del módem para obtener su dirección, máscara, gateway y DNS.

Después de obtener IPv4, el firmware comprueba si las redes AP y STA se solapan. Compara las direcciones de ambas interfaces bajo las dos máscaras. Por ejemplo, si el módem también utiliza `192.168.2.0/24`, la salida no puede distinguir correctamente esa red del segmento de clientes del ESP. También puede existir solapamiento con una máscara más amplia aunque los números de subred parezcan distintos.

Cuando detecta conflicto, conserva el dato de que la STA obtuvo IP, activa `subnet_conflict`, deja NAPT deshabilitado y retira el DNS de origen. No cambia automáticamente la IP del AP. Resolverlo requiere elegir subredes diferentes y ajustar la configuración de dirección en el código; el filtro de clientes DNS usa igualmente una máscara /24.

Si las redes son compatibles, se selecciona la STA como interfaz de salida predeterminada y se habilita NAPT sobre el AP mediante `esp_netif_napt_enable()` [2]. Los defaults comunes activan:

```
CONFIG_LWIP_IP_FORWARD=y
CONFIG_LWIP_IPV4_NAPT=y
CONFIG_LWIP_IPV4_NAPT_PORTMAP=y
```

NAPT mantiene traducciones de direcciones y puertos para los flujos de salida de los clientes. El módem ve ese tráfico a través de la dirección STA del ESP; las respuestas se devuelven al cliente que originó el flujo. No se necesita abrir un socket BSD de la aplicación por cada conexión reenviada: las traducciones pertenecen a la pila de red. Su tabla y sus búferes sí consumen recursos de lwIP.

Razón de diseño. Habilitar NAPT únicamente con IPv4 útil evita presentar como disponible una salida que todavía no tiene ruta. Deshabilitarlo al perder la STA hace coherente el estado publicado. Detectar la superposición evita atribuir a una contraseña o al DNS un problema de direccionamiento.

El soporte de port mapping está compilado, pero la aplicación no crea reglas de publicación de puertos ni proporciona una interfaz para ellas. Tampoco implementa encaminamiento IPv6. Consultar un registro DNS AAAA puede producir una respuesta válida sin que exista conectividad IPv6 para usarla.

7. Proxy DNS: la razón del DNS local estable

7.1. Por qué cambiar el DNS de DHCP no bastaba

Un teléfono puede asociarse al AP y obtener su concesión DHCP antes de que el ESP se conecte al módem. La concesión incluye un servidor DNS que el teléfono conserva. Modificar después la configuración del servidor DHCP no obliga al teléfono a solicitar una concesión inmediatamente. Si la dirección recibida no ofrece un DNS funcional, los dominios fallan aunque NAPT ya permita tráfico por dirección IP.

La implementación mantiene una dirección estable: DHCP anuncia desde el arranque `192.168.2.1` como DNS, y el proxy escucha allí en UDP53 antes de iniciar la radio. Cuando la STA obtiene conexión, solo cambia el servidor al que el proxy reenvía consultas. No se reinicia DHCP para intentar sustituir el DNS en concesiones existentes.

Razón de diseño. El contrato con el cliente permanece constante durante el aprovisionamiento. Esta separación entre DNS anunciado y DNS de origen resuelve el caso en que el teléfono solo podía navegar después de apagar y encender su Wi-Fi. El cliente conserva gateway y DNS, mientras el ESP actualiza su salida internamente.

7.2. Selección del servidor de origen

Al obtener IPv4 se consultan los DNS `MAIN`, `BACKUP` y `FALLBACK` de la STA, en ese orden. Se elige el primero que sea IPv4 y pase las comprobaciones del código: no ser la IP del AP, no pertenecer al primer octeto 0 ni 127, y estar por debajo del rango multicast. Si ninguno cumple, se utiliza `1.1.1.1` y se registra un aviso.

La selección es una búsqueda de configuración válida, no una comprobación de disponibilidad. El proxy no cambia automáticamente al segundo DNS cuando una consulta expira; tampoco prueba que la red permita alcanzar el DNS público de reserva. Una red con DNS interno o restricciones de salida puede necesitar ajustes.

7.3. Recorrido de una consulta

El proxy usa dos sockets UDP: uno enlazado a la IP del AP y puerto 53, y otro conectado al DNS de origen. Una tarea FreeRTOS con pila de 4096 bytes y prioridad 5 atiende las operaciones mediante `select()`.

1. Recibe el datagrama y comprueba su procedencia del segmento AP /24. Valida la cabecera, que sea una consulta ordinaria y que contenga una sola pregunta con nombre y campos dentro de los límites.
2. Reserva una de las seis entradas pendientes. Guarda el cliente, su identificador original, la pregunta necesaria para responder con error y el instante de inicio.
3. Reemplaza el identificador DNS por uno propio, único entre las entradas activas. El contador inicial parte de `esp_random()`. Reenvía el mensaje manteniendo el resto de su contenido.
4. Recibe desde el socket asociado al DNS seleccionado y comprueba que sea una respuesta para una transacción pendiente. La estructura de cabecera, ID y códigos de respuesta sigue el formato DNS [7].
5. Restaura el identificador original y entrega el mensaje al cliente.

Dos teléfonos pueden usar el mismo identificador DNS. La reescritura y la tabla de pendientes evitan confundirlos, incluso si las respuestas llegan en orden inverso. El socket conectado limita la recepción al par remoto configurado; la coincidencia de identificador es una comprobación de correlación y no sustituye autenticación criptográfica de DNS.

El socket de salida no se enlaza a la IP del AP: utiliza la ruta predeterminada STA. Enlazarlo a la dirección de la red de clientes introduciría una elección de origen inadecuada para el camino al módem. En UDP, `connect()` selecciona un par; no realiza un intercambio que demuestre que el servidor responde.

7.4. Límites, errores y cambios de conexión

Situación	Tratamiento actual
Consulta inválida	Respuesta `FORMERR` cuando puede construirse
Sin DNS de origen, tabla llena o fallo de envío	`SERVFAIL`
Sin respuesta en dos segundos	Expiración y `SERVFAIL` al cliente
Consulta mayor de 4096 bytes	`FORMERR`
Respuesta mayor de 4096 bytes	Indicador `TC`; sin fallback TCP local
Cambio de DNS o pérdida de STA	Invalida disponibilidad, falla pendientes y recrea el socket de origen
Respuesta con ID desconocido	Se descarta

Se usan hasta seis pendientes y un búfer de 4097 bytes para detectar mensajes que exceden el máximo aceptado de 4096. El proxy admite preguntas ordinarias con nombre sin compresión; no pretende interpretar todas las variantes posibles del protocolo. Los datos adicionales aceptados se reenvían sin convertir cada tipo de registro a una estructura propia. Por eso el transporte puede llevar preguntas A, AAAA o HTTPS sin que el firmware tenga que conocer el significado de cada respuesta.

Razón de diseño. `SERVFAIL` representa indisponibilidad o fallo temporal; `NXDOMAIN` afirmaría que el dominio no existe y podría dejar una respuesta negativa en caché. Los límites fijos acotan memoria y tiempo de retención en un microcontrolador. El precio es rechazar carga superior a seis pendientes y no cubrir todos los tamaños o transportes de DNS.

Marcar una respuesta como truncada evita entregar silenciosamente un mensaje incompleto. Sin embargo, un cliente que intente continuar por TCP en el puerto 53 no encontrará un servidor TCP en el proxy. Es una limitación real para respuestas grandes, no una implementación completa de fallback. No hay caché DNS, DoH, DoT ni validación DNSSEC propia. Los requisitos generales del transporte TCP están descritos en RFC 7766 [8].

Los cambios de servidor usan una dirección y un número de generación atómicos. `dns_ready` exige que el proxy esté iniciado y que el socket de origen corresponda a la generación vigente. Así no se conserva un estado preparado asociado al DNS de una conexión anterior. La tarea revisa actividad con un intervalo de selección de 100 ms y, cuando falla la creación del socket de origen, vuelve a intentarla aproximadamente cada segundo. Ninguno de esos indicadores demuestra por sí solo salida a Internet.

8. Servidor HTTP y aplicación del navegador

8.1. Recursos y contrato de las rutas

Las páginas `app/main.html` y `app/saved.html` se incorporan como texto al binario mediante `EMBED_TXTFILES`; el favicon se añade como binario con `EMBED_FILES`. El componente servidor usa los símbolos del enlazador para transmitirlos. En los HTML descuenta el NUL que añade la incorporación de texto; para el icono transmite el tamaño binario completo.

Razón de diseño. La aplicación es pequeña y estática. Embebiendo sus recursos se evita montar un sistema de archivos, gestionar una partición web o distribuir archivos por separado. Los recursos y su API viajan con la misma versión del firmware. Cambiar HTML exige recompilar y programar esa versión; no existe un cargador de páginas independiente.

Petición	Respuesta normal	Comportamiento relevante
`GET /`	HTML del formulario	Tipo UTF-8, `no-cache`, `nosniff`
`POST /`	`303` a `/saved`	Valida, guarda y encola aplicación de las credenciales
`GET /saved`	HTML de confirmación	`no-store`; redirige a `/` si no hay credenciales guardadas
`GET /api/status`	JSON de estado	`no-store`; no incluye SSID ni contraseña
`GET /favicon.ico`	Icono	Caché de un año, `immutable`, `nosniff`

Un POST inválido produce 400; un timeout al recibir el cuerpo, 408. Si no se puede guardar, se responde 500. Si se guardó pero falló la publicación del evento de aplicación, también se devuelve 500, con un mensaje que distingue que los datos quedaron persistidos y pueden aplicarse al reiniciar. La confirmación 303 se envía cuando ambas etapas inmediatas tuvieron éxito; no espera a que el módem acepte la contraseña.

Razón de diseño. El patrón POST–redirección–GET permite recargar la página de confirmación sin volver a enviar las credenciales. Mostrar primero el guardado y luego el progreso evita dejar al usuario sin respuesta durante el escaneo, DHCP y los reintentos. La autenticación Wi-Fi se informa de forma asíncrona mediante la API de estado.

8.2. Estado publicado y significado de cada campo

Una respuesta preparada puede ser:

```
{
  "credentials_saved": true,
  "connected": true,
  "nat_enabled": true,
  "dns_ready": true,
  "subnet_conflict": false,
  "wifi_reason": 0,
  "connection_error": "none"
}
```

Campo	Interpretación
`credentials_saved`	Hay credenciales cargadas válidas o el guardado actual terminó correctamente; no certifica su aceptación por el módem.
`connected`	La STA recibió IPv4 y todavía no se invalidó ese estado.
`nat_enabled`	La activación de NAPT devolvió éxito.
`dns_ready`	Proxy iniciado y socket UDP de origen preparado para la generación actual.
`subnet_conflict`	Se detectó superposición de las redes AP y STA.
`wifi_reason`	Última razón numérica Wi-Fi registrada, o cero sin diagnóstico vigente.
`connection_error`	Categoría estable derivada de esa razón para la interfaz.

La condición local de listo combina conexión, NAPT, DNS y ausencia de conflicto. No comprueba resolución efectiva en cada sondeo ni acceso a un sitio externo. Tampoco hay un watchdog de conectividad global que reinicie el dispositivo si el módem conserva Wi-Fi pero pierde Internet.

8.3. Sondeo automático y presentación

La página de confirmación consulta la API cada dos segundos durante la espera o el error, y cada diez segundos cuando está preparada. Cada petición tiene un límite de cinco segundos. Esta periodicidad corresponde al navegador que mantiene abierta la página; no es una tarea de monitorización adicional que siga ejecutándose dentro del ESP después de cerrar la pestaña.

El indicador de carga aparece en espera y desaparece en estado listo, error o inactivo. No existe ya un botón de actualización manual. Aunque desaparezca la animación al estar preparado, el sondeo más lento permite reflejar una pérdida posterior. Un fallo de petición HTTP no demuestra por sí solo que la contraseña sea incorrecta. Pasados treinta segundos sin un diagnóstico específico se muestra una espera prolongada. La preferencia de movimiento reducido del sistema desactiva la animación.

El botón del ojo utiliza SVG en línea, cambia el tipo del campo entre `password` y `text` y actualiza sus atributos accesibles. Es un botón de tipo `button`, de modo que usarlo no envía el formulario. Se desactivan autocorrección, capitalización automática y revisión ortográfica para reducir modificaciones involuntarias del dato.

Razón de diseño. Sondear un pequeño JSON mantiene sencilla la interacción con un servidor embebido y permite verificar el comportamiento en host. No requiere WebSocket ni una conexión de eventos permanente. El intervalo más largo al estar listo reduce peticiones; separar error de espera evita una animación indefinida que esconda el diagnóstico.

9. Diagnóstico de asociación y autenticación

El controlador informa razones de desconexión; `sta.c` las agrupa en categorías que la página transforma en mensajes comprensibles. Los códigos representativos siguientes no agotan todos los motivos de cada grupo; la tabla completa vigente está en `router_wifi_error_for_reason()`.

Categoría	Ejemplos	Interpretación práctica
`none`	0	Sin error Wi-Fi registrado en este momento.

Categoría	Ejemplos	Interpretación práctica
`network_not_found`	201	No se encontró el AP solicitado: revisar SSID, alcance y disponibilidad de la red.
`handshake_timeout`	15, 204	No terminó el intercambio de claves; una contraseña incorrecta es una causa posible.
`authentication_failed`	202, 23	Rechazo o fallo de autenticación; revisar clave y seguridad del módem.
`authentication_timeout`	2	Expiró la fase de autenticación.
`security_mismatch`	210, 211	Seguridad o cifrado incompatibles con la configuración aceptada.
`signal_too_weak`	212	No se encontró una red que cumpla el umbral de señal indicado por el controlador.
`association_failed`	203	Falló la asociación al AP.
`modem_busy`	5	El AP rechazó más estaciones asociadas.
`connection_lost`	200, 4	Pérdida de beacons, inactividad u otra salida de la red.
`connection_failed`	Otros	Fallo que no pertenece a una categoría más específica.

No se puede asegurar en todos los equipos que un timeout sea exclusivamente una contraseña incorrecta. Interferencias, pérdida de tramas o ajustes del módem pueden impedir completar el mismo intercambio. La página orienta hacia una causa probable y conserva el código de diagnóstico; no inventa una certeza que el controlador no proporciona. La ausencia de un mensaje exacto no es una política deliberada de ocultar el error por seguridad.

La última razón real permanece durante los reintentos automáticos, en lugar de borrarse cada tres segundos y hacer que el mensaje desaparezca. Se limpia al guardar nuevas credenciales o al alcanzar una asociación correcta. Las salidas locales esperadas al cambiar configuración se tratan por separado, como se explicó en la sección 4.

Razón de diseño. Una clasificación estable desacopla la interfaz de decenas de constantes del SDK. Conservar razón numérica y categoría permite una explicación breve al usuario y un dato más preciso para revisar el log. La persistencia temporal del diagnóstico mejora su visibilidad sin detener los intentos de recuperación.

10. Indicador y opciones específicas del hardware

El indicador es seleccionable en el menú **NAT Router**: sin LED, LED GPIO o RGB direccionable. El ESP32 utiliza por defecto GPIO2 activo bajo. El perfil S3 utiliza RGB en GPIO48, con brillo inicial 8. En ESP32-S3-DevKitC-1 v1.1 el RGB se trasladó a GPIO38; el pin debe ajustarse a la revisión física [5].

Sin IPv4 STA, el GPIO parpadea y el RGB parpadea en ámbar. Con IPv4, el GPIO permanece encendido y el RGB pasa a verde. La actualización se realiza cada 500 ms desde `app_main()`: un ciclo encendido/apagado completo en desconexión dura aproximadamente un segundo. El LED refleja `router_wifi_is_connected()`, no las tres condiciones de conexión, NAPT y DNS juntas. Puede verse verde aunque el módem no tenga Internet o exista conflicto de subred.

10.1. Por qué se utiliza RMT para el RGB

El LED direccionable requiere pulsos precisos. El código utiliza el controlador RMT de transmisión de ESP-IDF y un copy encoder [4]; no depende de temporización manual de GPIO dentro de una tarea ni de un componente externo de tiras LED. Configura una resolución de 10 MHz: cada tick dura 0,1 microsegundos.

Se construyen 24 símbolos para los bits de color en orden GRB y un símbolo adicional de reset. Para un cero se generan 3 ticks altos y 9 bajos; para un uno, 7 altos y 6 bajos. El reset ocupa 300 microsegundos en bajo. Los 25 símbolos caben en los 48 símbolos de memoria de un canal del S3, por lo que no se necesita DMA ni rellenar una trama más larga por bloques. La cola de transmisión tiene profundidad 1.

La trama se almacena estáticamente para mantener válido el búfer durante la transmisión, incluso si se agota la espera. Si el GPIO no es válido, falla la reserva del canal o se produce un error de actualización, se registra el problema y se desactiva el indicador.

Razón de diseño. El LED es una ayuda de observación y su fallo no debe impedir el router. RMT proporciona generación de pulsos por hardware mientras Wi-Fi y otras tareas están activas. Limitarse a una trama completa pequeña reduce la complejidad del codificador y evita exigir DMA solo para un LED.

10.2. FTM y consola USB

El menú incluye un responder FTM opcional, condicionado por la capacidad del target y las opciones del SDK. Permite responder a clientes Wi-Fi RTT compatibles; está desactivado en los perfiles base. No añade una interfaz de medición de distancia ni aumenta la velocidad del router.

El perfil S3 conserva la consola secundaria USB Serial/JTAG. Es una opción de observación y depuración del hardware, no una función web ni un intérprete de comandos remoto de la aplicación. PSRAM no se activa por suponer que todos los módulos S3 la incluyen: debe verificarse el módulo y su interfaz física.

11. Perfiles de ESP-IDF y compilación reproducible

11.1. Chip, placa, defaults y configuración generada

ESP32 y ESP32-S3 requieren seleccionar targets distintos. Además, un mismo chip puede montarse en placas con flash, PSRAM o LED diferentes. Los perfiles incluidos son preferencias de las placas del proyecto, no una descripción universal de todos los módulos de cada familia.

Archivo	Papel
`sdkconfig.defaults`	Forwarding IPv4, NAPT y soporte de port mapping comunes.
`sdkconfig.defaults.esp32`	DIO/40 MHz, flash declarada de 2 MB, GPIO2 activo bajo.
`sdkconfig.defaults.esp32s3`	DIO/80 MHz, flash de 16 MB, RGB48 y opciones específicas del S3 usado.
`sdkconfig`	Configuración activa completa generada por ESP-IDF.
`sdkconfig.old`	Respaldo automático previo; no es un perfil permanente de un chip.

ESP-IDF carga los defaults comunes y después el sufijo correspondiente a `IDF_TARGET` [6]. Una selección explícita ya guardada en `sdkconfig` tiene prioridad sobre defaults. Por eso editar un

perfil no fuerza necesariamente todos sus valores en una configuración ya existente. En IDF 6.1 las opciones con procedencia `# default:` pueden adoptar nuevos defaults durante la reconfiguración.

El campo `CONFIG_IDF_TARGET` identifica el chip de cada configuración generada. Al organizar estos perfiles, el archivo raíz activo correspondía a `esp32` y el respaldo a `esp32s3`; esos contenidos pueden cambiar al volver a configurar. El significado estable de `.old` es respaldo, no S3. No deben copiarse configuraciones completas entre chips ni editarse manualmente las banderas de capacidades `SOC_*`.

El perfil S3 conserva la consola USB secundaria, `CONFIG_FREERTOS_IN_IRAM`, colores del bootloader y colores del log. El tamaño de 16 MB se apoyó en la detección de flash del equipo probado. Los 2 MB del perfil ESP32 son el tamaño declarado en su configuración; no certifican la capacidad física de cualquier placa ESP32. 40/80 MHz indican reloj de la memoria flash, no la banda de radio Wi-Fi. El tamaño de flash tampoco aumenta por sí mismo la partición de aplicación: el esquema de particiones debe revisarse si se requiere una aplicación mayor.

Razón de diseño. Los sufijos estándar de ESP-IDF hacen explícitas las diferencias de placa sin convertir un respaldo generado en una segunda configuración oficial. Mantener lo común en el archivo base reduce divergencias de forwarding o NAPT entre chips. Conservar el GPIO configurable evita ligar la lógica de red a una revisión concreta del DevKit.

11.2. Comandos de trabajo

Desde una terminal con el entorno ESP-IDF 6.1 activado, el flujo habitual consiste en seleccionar un único target, configurar, compilar y programar:

```
# Elegir esp32 O esp32s3, segun el chip real.
idf.py set-target esp32s3
idf.py menuconfig
idf.py build
idf.py -p COM3 flash monitor
```

`COM3` debe reemplazarse por el puerto real. Se sale del monitor con `Ctrl+]`. `set-target` limpia el directorio de compilación seleccionado y regenera la configuración; no es un nombre alternativo para elegir la placa al programar un binario ya existente. Si el entorno define `IDF_TARGET`, debe coincidir con el target solicitado.

Para trabajar con ambos chips sin compartir configuración generada ni caché, pueden utilizarse carpetas de compilación separadas:

```
# PowerShell, desde la raiz del proyecto.
Remove-Item Env:IDF_TARGET -ErrorAction SilentlyContinue

idf.py -B build/esp32 -D IDF_TARGET=esp32 -D SDKCONFIG=build/esp32/sdkconfig build
idf.py -B build/esp32s3 -D IDF_TARGET=esp32s3 -D SDKCONFIG=build/esp32s3/sdkconfig build

# Programar especificamente el resultado S3.
idf.py -B build/esp32s3 -D IDF_TARGET=esp32s3 -D SDKCONFIG=build/esp32s3/sdkconfig -p COM3 flash
    monitor
```

Las mismas opciones deben acompañar a `menuconfig`, `build`, `flash` y `monitor`. Si una carpeta tiene caché de otro target o de otra ruta `SDKCONFIG`, debe elegirse una carpeta nueva o regenerarla de forma consciente.

`idf.py save-defconfig` escribe por defecto en el archivo raíz `sdkconfig.defaults`; no distribuye automáticamente las opciones entre los dos sufijos. Antes de conservar sus resultados deben

separarse preferencias comunes y de placa. La guía `CONFIGURACION.md` contiene los comandos de mantenimiento y las comprobaciones de los perfiles.

12. Presupuesto de recursos y tratamiento de fallos

Con la configuración revisada, lwIP tiene diez sockets disponibles. El servidor fija cuatro sesiones HTTP abiertas como máximo y admite purga de la sesión menos recientemente usada. Según la reserva interna del servidor de IDF, se presupuestan tres sockets adicionales de HTTP; el proxy usa dos. El total presupuestado es por tanto $4 + 3 + 2 = 9$, dentro del límite de diez.

Razón de diseño. Dejar el máximo HTTP predeterminado sin tener en cuenta DNS podría agotar los sockets al iniciar ambos servicios. El límite de cuatro sesiones y la purga LRU equilibran conexiones de navegador y el servicio DNS. No representa cuatro clientes Wi-Fi: una sola pestaña puede utilizar más de una conexión HTTP y el AP mantiene su límite independiente de tres estaciones.

Las seis entradas DNS y el búfer de paquetes son estáticos y pertenecen a la tarea DNS. Evitan alojar el paquete de unos 4 KB en su pila de 4 KB. Entre paquete y tabla pendiente se reserva del orden de 6 KB estáticos, además de la pila de tarea. Esto es una estimación de esas estructuras, no el consumo total de RAM: faltan TCB, sockets, pbufs, controlador Wi-Fi, servidor, otras tareas y alineación. La disponibilidad real se comprueba en el mapa de enlace y las métricas de heap.

El servidor reserva ocho posiciones de URI y registra cinco rutas. Los recursos HTML ocupan flash del firmware y se sirven desde sus símbolos embebidos. El código no impone una tarea ni una conexión nueva por cada flujo NAPT, pero los límites internos de la pila siguen condicionando la carga posible.

Los fallos se manejan según su importancia:

- La inicialización imprescindible de NVS, red y listener DNS usa comprobaciones de error que pueden detener el arranque ante un fallo.
- La pérdida del módem activa recuperación por eventos y temporizador, conservando el AP para corregir la configuración.
- Un fallo al iniciar HTTP se registra; no hay actualmente un supervisor que reintente automáticamente el arranque de ese servidor.
- Un error del LED desactiva el indicador y deja continuar la red.

13. Seguridad y alcance real de la aplicación

La seguridad del enlace inalámbrico y la administración web son capas distintas. El AP utiliza `WIFI_AUTH_WPA3_PSK` cuando se compila con `CONFIG_ESP_WIFI_SOFTAP_SAE_SUPPORT`; en caso contrario utiliza `WIFI_AUTH_WPA2_PSK`. En ambos casos el código requiere PMF. No es un modo mixto automático WPA2/WPA3 para todos los clientes. La STA fija un umbral mínimo WPA2 y `WPA3_SAE_PWE_BOTH`. Un cliente antiguo sin la seguridad o PMF necesarios puede no asociarse.

El servidor usa HTTP en puerto80, sin inicio de sesión propio, HTTPS ni protección CSRF implementada por la aplicación. No se enlaza exclusivamente a la dirección AP; puede resultar accesible también por la dirección STA desde la red del módem, según el acceso que permita esa red. Poder alcanzar la interfaz HTTP permite enviar credenciales nuevas; la contraseña del AP no es una autenticación administrativa adicional para solicitudes que lleguen por otros caminos.

Los perfiles no habilitan cifrado de NVS ni de flash. Guardar una cadena en NVS aporta persistencia, no confidencialidad frente a lectura física de la memoria. El firmware evita imprimir contraseñas en sus mensajes propios y la API no las publica, pero no incorpora un sistema de gestión de secretos, borrado seguro de todos los búferes temporales o control de acceso por usuario. La clave del AP inicial es conocida y está definida en `components/wifi/wifi_internal.h`.

El listener DNS sí está enlazado a la IP del AP y filtra procedencias de su /24. Estas comprobaciones limitan su alcance, pero no equivalen a DNS autenticado. La reescritura de IDs sirve para correlacionar transacciones; no convierte el protocolo en DNS cifrado.

Como futuras mejoras, podrían evaluarse autenticación administrativa, restricción de interfaces HTTP, protección de solicitudes de cambio, cifrado de almacenamiento con aprovisionamiento de claves y un formato versionado de credenciales. También pueden estudiarse DNS por TCP y failover de origen. Son propuestas, no funciones incluidas ni garantías de esta versión. OTA tampoco está implementado.

14. Verificación y diagnóstico práctico

14.1. Qué evidencia existe

Las validaciones registradas en el proyecto tienen alcances diferentes:

- **Compilación por target.** Se generaron configuraciones nuevas para ESP32 y ESP32-S3 con ESP-IDF 6.1 y se compilaron los dos perfiles. Esto comprueba dependencias, símbolos, selección de target y enlace de recursos. No equivale a programar ambas placas en cada revisión.
- **Parser en host.** Se compila el C de producción y se prueban límites, UTF-8, escapes, duplicados y entradas inválidas.
- **DNS en host.** El código de producción se adapta a sockets del host y se prueban siete escenarios con UDP real en loopback, puerto de prueba 1053: sin origen, preservación del mensaje, clientes con igual ID y respuestas invertidas, cambio de origen, respuesta desconocida, expiración y nueva pérdida de origen. El firmware usa el puerto 53.
- **Interfaz en host.** Se ejecuta el JavaScript real de la confirmación con DOM y peticiones simuladas: errores, reintentos, nuevas credenciales, recuperación y respuestas de estado inválidas.
- **Pruebas manuales reportadas en S3.** Se verificaron primer guardado, navegación desde el AP, SSID erróneo, contraseña errónea, recuperación con datos correctos y reinicio conservando NVS.

Los comandos se encuentran en `tests/README.md`. Las pruebas de host no emulan el controlador de radio, DHCP de lwIP, NAPT real ni el comportamiento de todos los teléfonos. No hay aquí una medición de throughput, latencia, consumo, carga máxima con tres estaciones o estabilidad durante días. No debe deducirse una cifra de rendimiento del éxito de compilación ni de una navegación manual.

14.2. Orden útil para localizar un problema

1. **Acceso local.** Comprobar asociación a `ESP32-NAT`, dirección obtenida por DHCP y acceso explícito por HTTP a `192.168.2.1`. No se implementa una redirección universal de portal cautivo ni una captura general de todos los dominios.

2. **Credenciales.** Confirmar que se recibió la página de guardado y revisar `connection_error`/`wifi_reason`. Un error 201 orienta a red no encontrada; 15/204, al intercambio de claves.
3. **Dirección de salida.** Revisar `connected` y el log de IPv4. Asociación sin IP sugiere revisar DHCP antes de atribuirlo a NAPT.
4. **Redes diferentes.** Revisar `subnet_conflict`; corregir la superposición antes de probar DNS o navegación.
5. **NAPT y resolución.** Revisar `nat_enabled` y `dns_ready`. Si una dirección externa es accesible pero los nombres no resuelven, estudiar el DNS de origen y sus restricciones.
6. **Servicio del módem.** Si todo lo local está preparado, comprobar que la red del módem realmente tenga Internet. Un LED verde no descarta una falla del proveedor.

Un `ECONNRESET` HTTP puede ser el cierre de una sesión por parte del navegador; por sí solo no demuestra caída Wi-Fi. En cambio, las razones de desconexión del controlador y los eventos IP permiten relacionar un fallo de radio con el estado de salida. No se necesita borrar NVS como paso rutinario de diagnóstico: hacerlo elimina la persistencia que se pretende verificar.

15. Cómo generar y mantener este documento

Este archivo es autónomo. En Overleaf se crea un proyecto en blanco, se sube `funcionamiento_tecnico.tex` y se selecciona como documento principal, o se copia su contenido en `main.tex`. El compilador debe ser **pdfLaTeX**. Diagramas, tablas y referencias están dentro del mismo archivo. No se requieren imágenes externas, BibTeX, `minted` ni ejecución de comandos externos. Una segunda compilación actualiza índice y referencias cruzadas si la herramienta no la realiza automáticamente.

Para mantenerlo alineado con el firmware, revisar especialmente `components/wifi/wifi_internal.h`, `components/wifi/wifi.c`, `components/wifi/sta.c`, `components/wifi/ap.c`, `components/server/server.c`, `components/dns_proxy/dns_proxy.c`, `components/nvs/nvs.c`, `components/status_led/status_led.c`, `app/saved.html` y los tres archivos de defaults. El README raíz es la guía de uso; este informe explica mecanismos y decisiones. Los comentarios y nombres citados deben actualizarse cuando cambien los contratos públicos, límites o condiciones de disponibilidad.

Las referencias siguientes describen las APIs y protocolos de soporte; la autoridad para los valores y funciones específicas de la aplicación es su código fuente. Las páginas de documentación `latest` pueden evolucionar; las compilaciones documentadas aquí corresponden a ESP-IDF 6.1.

Referencias

[1] Espressif Systems. *ESP32-S3 Wi-Fi Driver: Overview, configuración y canal compartido APSTA.* https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/wifi-driver/overview.html

[2] Espressif Systems. *ESP-NETIF Programmer's Manual: direccionamiento, DHCP y NAPT.* https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/network/esp_netif_programming.html

[3] Espressif Systems. *Non-Volatile Storage Library.* https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/storage/nvs_flash.html

[4] Espressif Systems. *Remote Control Transceiver (RMT).* https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/rmt.html

[5] Espressif Systems. *ESP32-S3-DevKitC-1 v1.1 User Guide: diferencias de revisión y LED.* https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html

[6] Espressif Systems. *Build System: Target-dependent sdkconfig defaults.* https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/build-system.html#target-dependent-sdkconfig-defaults

[7] P. Mockapetris. *RFC1035: Domain Names—Implementation and Specification.* Estructura de mensajes, identificadores, errores y truncamiento DNS. https://www.rfc-editor.org/rfc/rfc1035.html

[8] J. Dickinson et al. *RFC7766: DNS Transport over TCP—Implementation Requirements.* Referencia para el transporte TCP que este proxy todavía no implementa. https://www.rfc-editor.org/rfc/rfc7766.html